Lazy or Eager? When Post-Copy Process Restore Pays over a Network, and Streaming the Working Set

Utkarsh Maurya
Independent Researcher
utkarsh@kernex.sbs

***Abstract*—Restoring a checkpointed process on another machine can move the whole image first (eager) or start at once and fetch pages on fault (lazy, post-copy). Lazy restore is usually evaluated against an eager baseline that reads its image from local disk, which hides the cost the network adds to eager. We measure both honestly for CRIU process restore, with time to first response (TTFR) as the metric, over a grid of round-trip times (0–2 ms) and bandwidths from 100 Mb/s to 10 Gb/s, for a PyTorch inference server and a Redis-compatible key-value store. No fetch policy wins everywhere: demand paging wins only when bandwidth is scarce and RTT is near zero, sequential prefetch wins in the middle, and moving the whole image wins once bandwidth $\times$ RTT is large, which for Redis is 10 of 12 grid points. Each lazy policy pays roughly one RTT per demand fault, and that term decides the map. We then remove it: the page server pushes a recorded first-touch trace over its own connection at line rate while demand faults keep a separate connection. With a trace recorded on an earlier run this wins all 12 grid points for PyTorch (1.19–3.20$\times$ over the best existing policy) but only 6 of 12 for Redis, whose heap layout is not reproducible across runs on our x86 host. With a trace recorded on the same image, as REAP does locally, it wins all 24 grid points (1.22–5.75$\times$) and its TTFR comes within 1–10 % of the working set's transfer time at 100 Mb/s and 1 Gb/s. On two Raspberry Pi 5s over a real 1 Gb/s link, streaming cuts TTFR by 1.5–3.0$\times$ for PyTorch and 3.5–4.0$\times$ for Redis.**

I. Introduction

Checkpoint/restore moves running processes between machines: for load balancing, for draining a host, and increasingly for starting serverless functions and agent sandboxes from a warm snapshot. CRIU [1] offers two ways to restore on a remote host. Eager restore copies the image and then maps it. Lazy restore [2] maps the process at once with its memory registered with `userfaultfd` [3]; a handler fetches each page from a page server on the source the first time the process touches it.

The usual argument for lazy restore is that a process touches a small part of its memory before it can do useful work. That argument compares lazy restore over the network with eager restore from local disk. A real migration has no local image: eager restore must first move it, at image/bandwidth. Lazy restore pays one round trip per demand fault instead. Which cost is smaller depends on the image size, the working set, the bandwidth and the RTT, and no published study maps it for process restore. REAP [4], FaaSnap [5] and Spice [6] restore from local storage; AWS records chunk access order for SnapStart and container loading [7], [8] but has not published a regime analysis.

This paper makes four contributions.

1) **A regime map** (§IV). We measure eager-over-network, demand paging, sequential/stride prefetch and an adaptive prefetcher across four RTTs and three bandwidths. No policy wins everywhere. The losing policies are not slow because of bandwidth: each demand fault costs about one RTT, and the thousands of faults left after prefetching decide the outcome.
2) **Trace-ordered streaming** (§V). The page server pushes a recorded first-touch trace as one request over its own TCP connection, at whatever rate TCP achieves; the handler installs pages as they land. Demand faults keep their own connection. TTFR then depends on the working set's transfer time plus one RTT per page the trace missed.
3) **Where traces come from matters** (§VI, §VII). A trace from an earlier run of the same program is enough for PyTorch (98–99 % page overlap) but not for Redis on our x86 host (64.5 %), where allocator placement shifts whole pages between runs even with ASLR off. A trace recorded on the same image removes misses almost entirely and wins every grid point for both workloads.
4) **Real hardware** (§IX). On two Raspberry Pi 5s over a 1 Gb/s link the results hold and are, if anything, stronger, and one loopback effect, a large penalty for streaming the rest of the image through a shared FIFO, turns out to be an emulation artifact.

All measurements use `TCP_NODELAY` on every page-server socket. A companion report [9] shows that without it, Nagle's algorithm interacting with delayed ACKs inverts the measured effect of prefetching on this system; we take the corrected setup as given here.

II. Background and Related Work

Post-copy restore. Post-copy migration starts the target at once and pulls memory on demand; Hines et al. [10] added adaptive pre-paging for virtual machines on a gigabit LAN. CRIU implements post-copy for processes: `criu restore --lazy-pages` registers the restored memory with `userfaultfd` and hands the descriptor to a lazy-pages daemon, which resolves each fault with `UFFDIO_COPY`. An open CRIU pull request streams images for post-copy [11].

Snapshot cold starts. REAP [4] observed that serverless functions touch a stable working set after restore, records it on the first restore, and prefetches it on later ones. FaaSnap [5] refines this for VM snapshots. Spice [6] shows that with the right kernel primitives local process restore approaches

warm-start latency. These systems restore from local storage, where a page costs microseconds, not an RTT. Firecracker [12] documents the `userfaultfd` handling rules a snapshot handler must follow [13].

Network restore in industry. AWS Lambda loads container images and SnapStart snapshots on demand from a tiered cache and prefetches chunks in a recorded access order [7], [8]. That is the closest design to ours; the analysis of when it pays has not been published.

Far-memory prefetching. Leap [14] and 3PO [15] prefetch remote memory in steady state. Restore is a different problem: every page starts remote, and the access stream before the first response is short and mostly cold.

III. System and Methodology

Pipeline. The workload runs on the source, is checkpointed with CRIU, and is restored on the target. The page server is a Python TCP server that indexes the CRIU pagemap and serves 4 KiB pages by virtual address. The lazy-pages handler is a C daemon that owns the `userfaultfd`. It fetches the faulting page synchronously and can issue prefetches asynchronously off the fault path.

Policies. We compare:

- full: eager restore from images already on the target. This is a floor, not a contender: it moves no data.
- full-net: eager restore after streaming the whole image from the source over the same shaped link.
- lazy: demand paging only.
- prefetch: on each fault, also fetch the next pages sequentially, or along a detected stride.
- adaptive: prefetch with a controller that backs off when prefetched pages go unused.
- stream, stream-pf, stream-all, stream-same: trace-ordered streaming (§V).

Workloads. *PyTorch*: a ResNet-18 inference server; the probe writes a trigger file and waits for one inference result. Its image is 80.4k pages (329 MB) on x86, of which 15.5k (63 MB, 19 %) are touched before the first result. *Redis*: a key-value server holding 200k keys of 1 KiB; the probe issues a fixed set of 20k `MGET`s in batches of 100. On the x86 host the server binary is Valkey 9.1.2 (the distribution installs it as `redis-server`); on the Pis it is Redis 8.0.2. The x86 Redis trace is 29.8k pages (122 MB). Probes reach Redis over a Unix socket, so probe traffic is not shaped; the Pi kernel lacks `unix_diag`, so there the probe uses TCP over loopback.

Metric. TTFR is the time from the start of `criu restore` until the probe's first response. For full-net it includes the image transfer.

Testbeds. *x86*: one AMD Ryzen 7 7735HS laptop, Linux 7.2.8, CRIU 4.2.1. Source and target share the host; the network is loopback shaped with `netem` [16]: delay d on `lo` (so RTT $= 2d$) and a rate cap. The grid is RTT $\in \{0, 0.2, 1, 2\}$ ms $\times$ rate $\in \{100\,\text{Mb/s}, 1\,\text{Gb/s}, 10\,\text{Gb/s}\}$. *Pi*: two Raspberry Pi 5s on a direct 1 GbE link (base RTT 164 μs). Pi A runs and restores the workload; Pi B runs the page server. `netem` delay goes on A's egress only, so B's data keeps B's real qdisc. Pi A runs Linux 6.18.59; PyTorch runs used CRIU 4.1.1, Redis runs CRIU 4.2.1 (4.1.1 cannot dump the Redis TCP socket on this kernel).

TABLE I
TTFR (ms, median of 5) for existing policies on x86. Bold: best contender. full reads local images and is shown as a floor.

RTT	rate	full	full-net	lazy	prefetch	adaptive
PyTorch (image 329 MB, working set 63 MB)						
0	100M	207	26664	**6278**	7706	7898
0	1G	206	2899	1388	**873**	889
0	10G	212	509	800	**335**	341
0.2	100M	210	26749	9405	**7855**	8060
0.2	1G	213	2904	4544	1153	**1134**
0.2	10G	222	**521**	3966	628	606
1	100M	222	26763	–	**9005**	9096
1	1G	208	2928	–	2550	**2286**
1	10G	212	**549**	–	2188	1711
2	100M	220	26765	–	10853	**10464**
2	1G	210	**2982**	–	4414	3726
2	10G	215	**627**	–	4231	3223
Redis (Valkey 9.1.2, trace 122 MB)						
0	100M	179	22311	**12331**	23202	23906
0	1G	180	**2427**	2687	2527	2515
0	10G	190	**440**	1550	744	756
0.2	100M	182	22317	**18192**	24048	24656
0.2	1G	177	**2435**	8822	3520	3480
0.2	10G	185	**436**	7803	1909	1825
1	100M	175	**22330**	–	28254	27257
1	1G	175	**2464**	–	8756	7775
1	10G	174	**478**	–	7594	5779
2	100M	177	**22354**	–	34679	31579
2	1G	172	**2496**	–	15704	12980
2	10G	182	**533**	–	14800	10657

Statistics. Five iterations per cell, each with a fresh workload and checkpoint. We report medians. For the existing policies the population standard deviation is at most 3.6 % of the median (one outlier iteration puts full, a 200 ms floor, at 10 %). Streaming with a cross-run trace varies far more, up to 38 % for stream-all and 31 % for Redis stream, because the number of misses changes from run to run (§VII); every win we report for those modes holds for all five iterations, not just the median. stream-same stays under 5 % except in six cells at 1–10 Gb/s with one slow iteration. Demand-only lazy is skipped at 1 and 2 ms RTT on x86, where it exceeds the 30 s probe timeout.

IV. The Regime Map

Table I shows TTFR for the existing policies. *Data:* `eval/results/regime/, redis-v2/`.

No single policy wins. For PyTorch there are three zones. Demand paging wins only at 100 Mb/s with zero RTT, where every extra byte is expensive and faults are cheap. Prefetch or the adaptive variant wins the middle of the grid. Eager full-net wins once bandwidth $\times$ RTT is large: at 10 Gb/s from 0.2 ms and at 1 Gb/s from 2 ms. For Redis the eager zone covers 10 of 12 cells; demand paging wins the other two.

Prefetch overfetches and misses the random workload. For PyTorch, prefetch serves about 22k pages against a 15.5k-page working set (42 % extra), which is why it loses to demand paging at 100 Mb/s. For Redis, whose probe reads hash-table

buckets and values at random, sequential prefetch is worse than nothing at low RTT (23.2 s vs 12.3 s at 0/100 Mb/s), and it still leaves 6–9k demand faults on the Pis (§IX).

Each fault costs one RTT. PyTorch under demand paging takes 15.5k faults; raising the RTT from 0 to 0.2 ms adds 3.2 s, which is 15.5k times 0.2 ms. Prefetch leaves about 2k faults and gains 3.5 s from 0 to 2 ms RTT. Eager pays one bulk transfer and no per-fault RTT, which is why it wins at high RTT despite moving five times more data than the working set.

The gap. The cost a restore cannot avoid is moving the working set: 63 MB is 5.1 s at 100 Mb/s, 0.51 s at 1 Gb/s and 51 ms at 10 Gb/s. The best existing policy is 1.24× above that at 100 Mb/s and zero RTT, and 5.9× above it at 1 Gb/s and 2 ms. A policy that moved exactly the working set, at line rate, without per-fault round trips, would close the gap.

V. Trace-Ordered Streaming

Recording. A record run is a demand-only lazy restore in which the handler appends each fault address to a trace file, in fault order. The trace is the working set before the first response, ordered by first touch. For traces reused across runs the workload is launched with ASLR off (`setarch -R`) so that addresses are comparable between runs.

Streaming. At restore the handler opens a second connection to the page server and sends one request: a magic marker, a count and the list of trace addresses. The server answers with the pages back to back, in trace order, without further requests. TCP alone sets the rate. A receive thread installs each page with `UFFDIO_COPY` as soon as its 4 KiB are complete and publishes how far into the trace it has got.

Demand path. Faults are served on the original connection, so a fault never waits behind the stream at the client. The handler classifies each fault against the stream: *landed* (the stream already delivered it; the fault raced the install), *ahead* (in the trace, not yet delivered) or *miss* (not in the trace). Ahead and miss pages are fetched on demand; an install that loses to the stream fails with `EEXIST` and is ignored.

Variants. stream uses a trace recorded on an earlier run. stream-pf adds sequential/stride prefetch, but only around misses. stream-all follows the trace with every other image page, ordered by distance to the nearest traced page. stream-same first runs an untimed record pass on the same image and then restores it again, streaming that trace; it models a system that records once per snapshot and restores it many times, as REAP does.

What it took. A first version reused the prefetch thread: one 8 B request per page, up to 1024 in flight. The Python server then capped at 44k pages/s, and at 2 ms RTT the request window let the process overtake the stream, so most faults were "ahead". Server push removed both: the server reaches about 275k pages/s on loopback at 10 Gb/s and line rate at 1 Gb/s, and the 0 ms/10 Gb/s PyTorch cell fell from 405 to 129 ms.

TABLE II
STREAMING VS THE BEST EXISTING POLICY ON X86 (TTFR, MS). "BEST OLD": BEST OF TABLE I. "CROSS-RUN": BEST OF STREAM, STREAM-PF, STREAM-ALL WITH A TRACE FROM AN EARLIER RUN. SPEEDUPS OF STREAM-SAME OVER BEST OLD.

		PyTorch				Redis		
RTT	rate	old	cross	same	×	old	cross	same
0	100M	6278	5274	**5129**	1.22	12331	13234	**9855**
0	1G	873	585	**546**	1.60	2427	1814	**1011**
0	10G	335	129	**93**	3.60	440	331	**131**
0.2	100M	7855	5292	**5136**	1.53	18192	14632	**9860**
0.2	1G	1134	682	**544**	2.08	2435	2506	**1024**
0.2	10G	521	163	**92**	5.66	436	331	**129**
1	100M	9005	5751	**5130**	1.76	22330	22023	**9875**
1	1G	2286	820	**554**	4.13	2464	2569	**1046**
1	10G	549	249	**99**	5.55	478	357	**140**
2	100M	10464	5679	**5147**	2.03	22354	24372	**9899**
2	1G	2982	1028	**561**	5.32	2496	2647	**1073**
2	10G	627	338	**109**	5.75	533	437	**214**

VI. Streaming Results on x86

Table II and Fig. 1 summarize the streaming modes. *Data:* `regime-stream/`, `redis-v2/`, `redis-same/`, `pytorch-same/`.

PyTorch, cross-run trace. A stream mode wins all 12 cells, by 1.19–3.20× over the best existing policy. Plain stream is best at 100 Mb/s, stream-pf in the middle, and stream-all at 10 Gb/s with RTT ≥ 1 ms. TTFR still rises with RTT (585 to 1028 ms at 1 Gb/s) because a few hundred faults miss the trace.

Redis, cross-run trace. Only 64.5 % of one recording's pages appear in the other. A stream mode beats the best existing policy by 1.22–1.34× in 6 cells, ties within 2 % in one, and loses the other five by 3–9 %: thousands of misses at one RTT each eat the gain.

Same-image trace. stream-same wins all 24 cells: by 1.22–5.75× over the best existing policy for PyTorch and 1.25–3.41× for Redis, and by 1.03–3.10× and 1.34–2.57× over the best cross-run stream. Demand faults fall to 0–4 misses for PyTorch and under 120 faults of any class for Redis. TTFR becomes nearly flat in RTT (PyTorch at 1 Gb/s: 546 to 561 ms over 0–2 ms) and tracks bandwidth. It lands within 1–2 % of working set/bandwidth at 100 Mb/s, 4–10 % at 1 Gb/s, and 30–120 ms above it at 10 Gb/s, where restore overheads dominate a 50–100 ms transfer.

The cost moved to the record pass. The record pass is a demand-only lazy restore. For PyTorch it takes 0.8–6.3 s at zero RTT and 32–38 s at 2 ms; for Redis about 65 s at 2 ms. stream-same is therefore a number for the second and later restores of a snapshot, not the first. Systems that restore one snapshot many times, such as serverless platforms and agent sandboxes, amortize it; a one-off migration cannot, and must use a cross-run trace or an existing policy.

VII. Why Cross-Run Traces Miss

Every miss costs one RTT on the demand connection, so the trace's coverage of the next run decides how close a cross-run stream gets to stream-same.

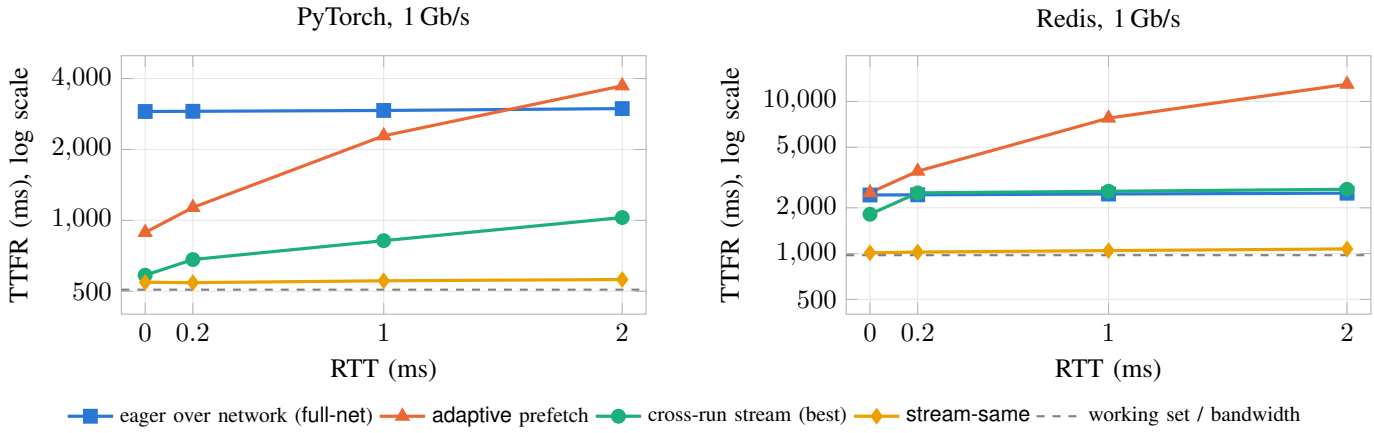


Fig. 1. TTFR against RTT at 1 Gb/s on x86. Lazy prefetching grows by about one RTT per remaining fault; eager pays for the whole image once; same-image streaming stays within 10 % of the working set's transfer time at every RTT.

PyTorch: fresh heap. Two PyTorch recordings share 98.0–99.0 % of pages with a median rank shift of 4. The differences sit in the `brk` heap: one run had a 617-page heap region the other lacked. Even the union of three runs' traces leaves 85–690 misses against a fourth. Every run allocates some fresh heap that no earlier trace can name.

Redis on x86: layout jitter. The two x86 Redis traces have the same number of pages in each 256 MB region (27,210 vs 27,212), but the long runs of pages sit a few pages apart: a 1313-page run starts at `...2f8e000` in one trace and `...2f7b000` in the other. The process maps are identical under `setarch -R`; placement inside the allocator's arenas differs. We started the server three times with ASLR off, loaded the same 200k keys, and read the addresses of 201 sampled keys with `DEBUG OBJECT`:

setup	keys at the same address
x86, Valkey 9.1.2, jemalloc 5.4.0	64–183 / 201
hash seed fixed	154–182 / 201
jemalloc 5.3.1 swapped in	142–156 / 201
pinned to 4 CPUs / 1 CPU	65–135 / 25–146
Pi, Valkey 8.1.1, jemalloc 5.3.0	197–201 / 201
Pi, Redis 8.0.2	193–199 / 201

The mismatches are whole-page offsets (4, 8 or 20 KB) that appear and vanish along the insertion order rather than accumulating, consistent with freed multi-page extents being reused in a timing-dependent order. Valkey's random hash seed contributes (its hash table allocates overflow buckets depending on where keys hash), and the allocator build contributes; disabling active rehashing or jemalloc's decay-based purging does not help. We did not isolate the rest, nor separate Valkey 9.1 from 8.1. The same server family is reproducible on the Pi, where the cross-run trace overlaps 98.8 % and cross-run streaming comes within 4 % of stream-same (§IX).

Implication. Cross-run traces work only for programs whose heap layout is reproducible, and that is a property of the build and platform, not just of the program. A same-image trace is immune: its addresses come from the image itself.

TABLE III
PI PYTORCH, STREAM AND STREAM-ALL WITH PI B'S QDISC `FQ_CODEL` VS `PFIFO`: TTFR (MS) AND MEDIAN MS PER MISS.

mode	delay	fq_codel	pfifo	ms/miss fq	ms/miss pfifo
stream	0	728	718	0.88	0.91
stream	1000	1127	1071	1.58	1.62
stream	2000	1594	1405	2.48	2.54
stream-all	0	1145	1127	1.99	2.25
stream-all	1000	1249	1178	2.50	2.63
stream-all	2000	1372	1292	3.53	3.56

VIII. Streaming the Rest of the Image

stream-all covers misses by streaming every remaining page after the trace. It cuts PyTorch faults to about 50 and wins at 10 Gb/s with high RTT, but on loopback it loses badly at 100 Mb/s: 55 misses cost 12.4 s, about 225 ms each, because each demand reply queues behind the stream at the shaped bottleneck. Delay-based congestion control (Vegas) on the stream socket or pacing the rest at 0.7–0.85× the measured delivery rate roughly halves the damage; TCP-LP does nothing, because demand traffic is too small to count as a competing flow. No endpoint-only variant matches plain stream at 100 Mb/s.

On the Pis, the same comparison shows a penalty of about 1.1 ms per miss, not 225 ms, and it is unchanged when Pi B's root qdisc is switched from `fq_codel` to a plain `pfifo` (Table III). We believe TCP Small Queues bounds how much of the sender's data waits in the qdisc and driver, so the queue a demand reply waits behind is a millisecond or two at line rate, whereas `netem` on loopback holds a deep emulated queue. We did not verify the TSQ explanation directly. The loopback FIFO penalty should be read as an emulation artifact, not an estimate for a real link.

IX. Real Hardware: Two Raspberry Pi 5s

Table IV repeats the comparison on two Raspberry Pi 5s. *Data:* `pi-stream/`, `pi-stream-pfifo/`, `pi-redis/`.

The loopback result holds. For PyTorch, stream-pf wins every delay by 1.46–2.96×, close to the x86 1 Gb/s cells

TABLE IV
PI TTFR (MS, MEDIAN OF 5) OVER A REAL 1 GBE LINK. RTT IS THE 164 μS BASE PLUS THE ADDED DELAY. SPEEDUP: BEST STREAMING MODE OVER THE BEST EXISTING POLICY. PYTORCH: IMAGE 74.7K PAGES, CRIU 4.1.1. REDIS 8.0.2: IMAGE 437 MB, TRACE 111 MB, CRIU 4.2.1.

mode	0.16 ms	1.16 ms	2.16 ms
PyTorch			
full (floor)	277	269	275
full-net	2939	2937	2947
prefetch	1023	2850	5014
adaptive	1029	2569	4226
stream	728	1127	1594
stream-pf	**700**	**867**	**1076**
stream-all	1145	1249	1372
speedup	1.46×	2.96×	2.74×
Redis			
full (floor)	259	260	258
full-net	4059	4064	4073
lazy	4586	–	–
prefetch	3606	9989	17709
adaptive	3616	8218	13433
stream	1018	1155	1359
stream-pf	1021	1049	1058
stream-all	1317	1715	1742
stream-same	**1002**	**1006**	**1014**
speedup	3.60×	4.04×	4.02×

(1.49–2.90×), and the existing policies order the same way: prefetch wins at low RTT and eager from about 2 ms. Each miss costs about one RTT plus 0.7 ms of service time on the Pi; prefetching around misses removes a third of them, which is where stream-pf's lead over stream comes from.

Redis gains most. Redis touches 111 MB of a 437 MB image before its first reply. Eager moves all of it (about 3.5 s at line rate); streaming moves the working set. Streaming wins by 3.5–4.0× at every delay. stream-same sits at the bandwidth floor: 111 MB at 1 Gb/s is about 0.9 s, it took 1.00–1.01 s, the process overtook the stream about 400 times ("ahead" faults) and only 7 pages missed. Here the cross-run trace overlaps 98.8 %, so stream-pf comes within 2–4 % of stream-same; the gap that mattered on x86 is absent.

Old lazy policies collapse with RTT. Redis reads its hash table at random, so prefetch guesses badly and still leaves 6–9k faults: the adaptive prefetcher reaches 13.4 s at 2.16 ms RTT, more than three times the eager time.

X. IMPLEMENTATION PITFALLS

Three problems cost us days and are easy to repeat.

`UFFDIO_COPY` returns `EAGAIN` and leaves the thread asleep. While a `REMOVE` or `UNMAP` event is pending (jemalloc returns memory with `MADV_DONTNEED`), the kernel refuses `UFFDIO_COPY` and `UFFDIO_ZEROPAGE` with `EAGAIN` and does not wake the faulting thread. A handler that treats the error as "someone else will install it" hangs the process; our Redis runs stalled for hours. The fix is to issue `UFFDIO_WAKE` on that range after `EAGAIN` so the thread refaults.

Image formats drift. CRIU 4.2 writes the pagemap entry's page count as a new 64-bit field and leaves the old one zero; an older `pycriu` then reads every entry as empty and the page server serves zeros. We fall back to decoding the field by hand.

Restoring an image twice. stream-same restores one image twice. The first restored process appends to its stdout log, and CRIU refuses to restore a process whose open file has changed size; CRIU 4.2 also refuses to overwrite an existing pidfile. The harness truncates the log to its dump-time size and removes the pidfile between passes.

XI. DISCUSSION AND LIMITATIONS

What to deploy. If a snapshot is restored more than once, record its first-touch trace on the first restore and stream it on later ones: TTFR then approaches the working set's transfer time at every RTT we tested. Cross-run traces are worth trying only where the program's heap layout is reproducible; checking two recordings' overlap is cheap and predicts the outcome. Without a trace, the regime map applies: demand paging at low bandwidth and near-zero RTT, prefetching in the middle, and eager transfer once bandwidth × RTT is large or the access pattern is random.

Limitations.

- Two workloads. Both touch a quarter of their image or less before the first response (19 % for PyTorch on x86, 25 % for Redis on the Pi); a workload that touches most of its image leaves streaming little to save over eager.
- x86 results use `netem` on loopback, with source and target on one host. The Pi results show one loopback effect (§VIII) does not transfer; others might not either.
- The page server is single-threaded Python. At 10 Gb/s its throughput and restore overheads, not the network, set the remaining 30–120 ms.
- Five iterations per cell. Cross-run streaming is noisy (standard deviation up to 38 % of the median); more iterations would narrow its medians, though not change which policy wins.
- TTFR only. We do not measure throughput after the first response, during which a lazy restore is still paging.
- The Pi PyTorch and Redis runs used different CRIU versions (4.1.1 and 4.2.1); we did not run stream-same for PyTorch on the Pi.
- The cause of x86 heap-layout jitter is only partly identified.

XII. CONCLUSION

Against an honest eager baseline that pays for moving its image, lazy restore is not a free win: for a random-access key-value store, eager transfer beats every lazy policy at 10 of 12 network settings we tested. What decides the map is one round trip per demand fault. Streaming a recorded first-touch trace by server push removes most of those round trips. With a trace recorded on the same snapshot, it beats every existing policy at all 24 settings, by up to 5.75×, and stays within 10 % of the time to move the working set at 100 Mb/s and 1 Gb/s. On real hardware it cuts TTFR by up to 4×.

Artifact Availability

Code, harnesses and raw results are in the DistriProc repository (`src/`, `eval/`); each section names its result directory under `eval/results/`.

References

[1] "CRIU: Checkpoint/restore in userspace," https://criu.org, 2013, project documentation.

[2] "CRIU lazy migration," https://criu.org/Lazy_migration, 2017, design wiki.

[3] "userfaultfd(2) — Linux programmer's manual," https://man7.org/linux/man-pages/man2/userfaultfd.2.html, 2016.

[4] D. Ustiugov, P. Petrov, M. Kogias, E. Bugnion, and B. Grot, "Benchmarking, analysis, and optimization of serverless function snapshots," in *Proc. ASPLOS*, 2021, pp. 559–572.

[5] L. Ao, G. Porter, and G. M. Voelker, "FaaSnap: FaaS made fast using snapshot-based VMs," in *Proc. EuroSys*, 2022.

[6] Holmes *et al.*, "Rethinking process snapshots for near-warm serverless cold starts," arXiv:2509.14292, 2025, to appear at OSDI 2026.

[7] "Under the hood: how AWS Lambda SnapStart optimizes function startup latency," https://aws.amazon.com/blogs/compute/under-the-hood-how-aws-lambda-snapstart-optimizes-function-startup-latency, aWS Compute Blog.

[8] M. Brooker, M. Danilov, C. Greenwood, and P. Piwonka, "On-demand container loading in AWS Lambda," in *Proc. USENIX ATC*, 2023.

[9] U. Maurya, "The prefetch paradox that wasn't: Four measurement artifacts in a CRIU lazy-restore study," 2026, companion report, under submission.

[10] M. R. Hines, U. Deshpande, and K. Gopalan, "Post-copy live migration of virtual machines," *ACM SIGOPS Operating Systems Review*, vol. 43, no. 3, pp. 14–26, 2009.

[11] "CRIU pull request #3200: image streaming for post-copy restore," https://github.com/checkpoint-restore/criu/pull/3200, open as of November 2026.

[12] A. Agache, M. Brooker, A. Iordache, A. Liguori, R. Neugebauer, P. Piwonka, and D.-M. Popa, "Firecracker: Lightweight virtualization for serverless applications," in *Proc. USENIX NSDI*, 2020.

[13] "Handling page faults on snapshot resume," https://github.com/firecracker-microvm/firecracker/blob/main/docs/snapshotting/handling-page-faults-on-snapshot-resume.md, firecracker documentation.

[14] H. Al Maruf and M. Chowdhury, "Effectively prefetching remote memory with Leap," in *Proc. USENIX ATC*, 2020.

[15] C. Branner-Augmon, N. Galstyan, S. Kumar, E. Amaro, A. Ousterhout, A. Panda, S. Ratnasamy, and S. Shenker, "3PO: Programmed far-memory prefetching for oblivious applications," arXiv:2207.07688, 2022.

[16] S. Hemminger, "Network emulation with NetEm," in *Linux Conf Au (linux.conf.au)*, 2005.